Certifiable Support Budgets versus Intrinsic Support in Quantum Compilation

Sze Chun Yiu

Abstract

A support bound can be exact for a restricted proof system and still be loose as a description of the compiler it certifies. We make that distinction explicit in two quantum-compilation families. The generic zero-sum language is donor mathematics: for a finite abelian group `H` and fixed alphabet `A`, let $\mathrm{zsf}(H; A)$ be the maximum length of a zero-sum-free sequence over `A`. In the registered rank-only deletion proof system, $\mathrm{zsf}(H; A)$ is the exact terminal support budget when a maximum zero-sum-free word is realizable and no additional rule is available.

The compiler quantity is different. Define intrinsic support $kappa(F, C)$ as the least `k` for which every instance of compiler family `F` under objective `C` has an exact optimum of support at most `k`. Evidence that `k-1` cannot suffice is required to establish an exact numerical value, but it is not part of the definition. This makes *kappa* a mathematical property of the compiler rather than a statement about current knowledge.

The main result is the matched tight/loose pair. In frozen one-Tag R6M, the rank-only certifiable support budget is two and independent compiler upper/lower evidence gives $kappa_R 6M = 2$. In frozen R6I under the same declared unit-objective interpretation used by its parent theorem, the rank-only system has exact budget five, while a stronger whole-system Tag-relocation theorem gives $kappa_R 6I = 1$. Thus

$beta_r ank - only(R6I) = 5 > 1 = kappa_R 6I$.

The separation is deliberately proof-system-relative: it does not prove that every local, syndrome-preserving, or unrestricted proof system needs support five. Registered direct products amplify the same mechanism to budgets `5t` versus `t` because the product definition imposes additive support and forbids cross-component moves; the corresponding $\Theta(n^{\lfloor 4t})$ direct-enumeration ratio is a corollary of that declared search model, not an algorithm-independent lower bound. The contribution is therefore the production tight/loose control pair and the identification of the missing operation in the looser certificate language, not a new generic zero-sum or proof-complexity theory.

Keywords: quantum compilation; certifiable support; intrinsic support; proof systems; Pauli strings

1 1. Scientific question and novelty boundary

Three quantities are often collapsed in compiler papers:

1. the smallest support the compiler intrinsically needs;
2. the support reached by a named normalization;
3. the support a restricted proof system can certify.

This paper asks when those quantities coincide and when they do not.

The generic ingredients—zero-sum thresholds, rank obstructions, proof-system-relative lower bounds, and direct products—are prior mathematics. The paper-level residual is:

- a tight production control in R6M;
- a strict production separation in R6I under an explicitly named rank-only proof system;
- the compiler operation absent from that proof system: whole-system Tag relocation/reconstruction.

Product amplification is reported only as a transparent corollary of the registered product construction.

2 2. Three support quantities

Fix a compiler family `F` and objective `C`.

2.1 2.1 Intrinsic support

Define

$kappa(F, C) = mink : everyinstanceinFhasanexactoptimumof$ support $\leq k$,

whenever such a uniform finite bound exists.

This is a mathematical property of `(F,C)`. To prove $kappa(F, C) = k$, one needs both an upper theorem at `k` and a lower witness showing that `k-1` fails for at least one admitted instance. The witness is evidence for the value; it is not part of the definition.

2.2 2.2 Normalization ceiling

A transformation `N` may show that every feasible configuration has a no-more-expensive representative of support at most `B_N`. Unless a matching intrinsic lower witness is known, `B_N` belongs to `(F,C,N)` rather than to the compiler alone.

2.3 2.3 Certifiable support budget

Let `P` be an explicitly registered proof system with a fixed observation language and legal rules. Define $beta_P(F, C)$ as the least uniform support budget that `P` can certify over the production scope for `(F,C)`.

This paper uses **certifiable support budget** rather than unqualified "certificate complexity" to avoid collision with the established Boolean-function quantity usually denoted `C(f)`.

Soundness of `P` gives

$$kappa(F, C) \leq beta_P(F, C)$$

when both quantities refer to the same compiler family and the same objective. Equality requires additional mathematics.

3 3. Donor zero-sum scaffold

Let `H` be a finite abelian group and let `A subseteq H` be fixed. A subsequence means an arbitrary nonempty subset of positions, not necessarily a contiguous factor. A sequence is zero-sum-free when no nonempty subsequence sums to zero. Define

$\mathrm{zsf}(H; A) = max(0 \cup |W| : Wiszero - sum - freeoverA)$.

Consider the abstract deletion language whose states are nonzero-total sequences over `A` and whose only legal shortening removes a nonempty **proper** zero-sum subsequence.

Lemma 1 (terminal length of the registered deletion language). The maximum terminal length is $\mathrm{zsf}(H; A)$.

Proof. Any sequence longer than zsf contains a nonempty zero-sum subsequence. Because the state's total sum is nonzero, that subsequence cannot be the whole sequence, so the deletion is legal. Conversely, a maximum zero-sum-free sequence has nonzero total: if its total were zero, the whole sequence itself would be a forbidden nonempty zero-sum subsequence. It therefore belongs to the state language and has no legal deletion. □

This lemma is generic zero-sum mathematics and is not claimed as a new theorem of this paper.

A production compiler obtains an exact zsf-based lower witness only when it realizes such a terminal word **and** the named proof system has no other rule that can reduce it.

4 4. Tight control: R6M

All quantities in this section use the frozen R6M unit objective bound by the parent evidence.

The one-Tag R6M production signature alphabet contains a basis of `F_2^2`. In the registered rank-only proof system, the basis word is terminal and the binary dependence upper bound is two, hence

$beta_rank - only(R6M) = 2.$

Separately, the all-size compiler normalization supplies support at most two and the committed lower witness rules out support one for an admitted instance. Therefore

$kappa_R6M = 2.$

Thus R6M is a genuine tight control:

$beta_rank - only(R6M) = kappa_R6M = 2.$

The proof-system certificate is not automatically loose; tightness depends on whether the certificate obstruction is also a compiler obstruction.

5 5. Strict separation: R6I

All quantities in this section use the frozen R6I unit objective named in the live claim ledger.

The R6I production block-deletion alphabets realize basis obstructions in a five-dimensional binary quotient. The explicitly registered **rank-only** proof system contains only the zero-sum/rank deletion rule for this bound. Therefore the production upper and realized terminal witness give

$beta_rank - only(R6I) = 5.$

This exactness is intentionally relative to that proof system. The paper does not assert that rank-only is the strongest proof language available to practitioners or that every local proof system has the same lower bound.

A stronger compiler transformation changes the auxiliary structure that rank-only freezes: it localizes each block to an anticommuting core and then relocates/reconstructs the shared Tag at whole-system scope. The committed all-size theorem gives support at most one, while support zero is infeasible. Therefore

$kappa_R6I = 1.$

The exact production separation is

$beta_rank - only(R6I) - kappa_R6I = 4.$

This identifies the missing proof operation rather than merely observing a smaller bound.

6 6. Registered product amplification

Let `F_R6I^t` be the registered direct product of `t` independent R6I components on disjoint coordinates. By definition:

1. support budgets add across components; and
2. the registered product proof system permits no cross-component move.

Theorem 2 (declared-product budgets). For every $t \geq 1$,
$beta_rank - only(F_R6I^t) = 5t$,
$kappa(F_R6I^t) = t$.

Proof. Componentwise upper bounds give $beta \leq 5t$ and $kappa \leq t$. For the rank-only lower bound, each component realizes its support-five terminal obstruction; because the product proof system has no cross-component move and the budget is additive, these `t` component lower bounds compose to $beta \geq 5t$. For intrinsic support, support zero is infeasible in every component, so any exact product solution needs at least one supported coordinate per component, giving $kappa \geq t$. The componentwise support-one compiler normalization attains `t`. □

The additive gap is `4t`. This is amplification of one registered mechanism, not a second independent compiler phenomenon.

6.1 6.1 Direct enumeration corollary

For a direct support enumerator over `n` coordinates with fixed local alphabet and fixed support budget `B`, the leading search volume is $\Theta(n^B)$. Under the declared product model the rank-only and intrinsic budgets therefore induce
$\Theta(n^(5t))$ and $\Theta(n^t)$
for fixed `t`, with ratio $\Theta(n^(4t))$ as `n -> infinity`.

The separate statement that the additive budget gap grows without bound is a `t -> infinity` observation. The two limits are not conflated, and neither statement is an algorithm-independent complexity-class lower bound.

7 7. Relation to prior work

Davenport and restricted zero-sum theory own the generic sequence thresholds. Sparse integer optimization owns general support bounds and lower constructions. Proof-complexity and formal-methods literature own the distinction between an object's property and what a restricted calculus can prove.

The substantive residual here is the **matched production pair**: R6M, where the rank-only certificate is tight, and R6I, where the same style of certificate is loose by five-to-one because the compiler admits a stronger global Tag reconstruction that the proof language freezes out.

The abstract zero-sum lemma and the declared-product search exponent are supporting scaffolds, not standalone novelty claims.

8 8. Reproducibility and evidence authority

The R6I production alphabets, basis witnesses, source/generic/native agreement and support-one parent theorem are commit-bound in the existing parent evidence package. The R6M support-two

upper theorem and support-one obstruction are commit-bound in the parent evidence package. Finite verifiers check representative alphabet and product identities; all-size authority for the compiler values comes from the corresponding parent theorems and witnesses.

Internal independent implementations are not external specialist replication.

9 9. Limitations

1. $beta_rank - only$ is exact only for the explicitly registered rank-only proof system.
2. No lower bound is proved for every local, syndrome-preserving or unrestricted proof system.
3. The product theorem uses additive support and no cross-component move by definition.
4. The search exponent concerns a direct support enumerator, not arbitrary algorithms.
5. Structural support is not a hardware speedup or physical quantum-resource advantage.
6. The generic zero-sum and proof-system concepts are donor-owned.
7. Submission-date overlap review and independent specialist proof attack remain external scientific checks.

10 10. Conclusion

A support ceiling should be assigned to the layer that proves it. In R6M, the rank-only ceiling and intrinsic compiler support coincide at two. In R6I, the rank-only budget is five while the compiler's intrinsic support is one because a stronger whole-system Tag transformation leaves the certificate language. The paper's contribution is this production tight/loose control pair and the identified missing proof operation. Generic zero-sum mathematics and direct-product amplification are supporting tools, not the novelty claim.

11 Tool-use disclosure

A generative language model assisted manuscript organization, language revision, adversarial review, and submission-package preparation. The listed author remains responsible for the mathematical statements, proofs, references, executable claims, and final text.

12 Data and code availability

The source package contains finite control records and the standalone verifier used for the dependent local lemmas and product bookkeeping. The all-size support claims remain proof- and witness-authorized; packaged computations do not enlarge them.

13 Author contributions

Sze Chun Yiu is the sole listed author and performed the authorship contributions represented in this manuscript, including the formal analysis, computational verification, and manuscript preparation.

14 References

1. I. Aliev, J. A. De Loera, F. Eisenbrand, T. Oertel, and R. Weismantel, "The Support of Integer Optimal Solutions," *SIAM Journal on Optimization* **28**, 2152–2157 (2018). DOI: 10.1137/17M1162792.

2. M. Freeze and W. A. Schmid, "Remarks on a generalization of the Davenport constant," *Discrete Mathematics* **310**, 3373–3389 (2010). DOI: 10.1016/j.disc.2010.07.028.

3. G. Wang, "The universal zero-sum invariant and weighted zero-sum for infinite abelian groups," *Communications in Algebra* **53**(4), 1581–1599 (2025). DOI: 10.1080/00927872.2024.2418017.

4. S. A. Cook and R. A. Reckhow, "The relative efficiency of propositional proof systems," *Journal of Symbolic Logic* **44**(1), 36–50 (1979). DOI: 10.2307/2273702.

5. N. Schillo, A. Sturm, and R. Quay, "TARE: Block Encoding Linear Combinations of Pauli Strings Without Ancilla State Preparation," arXiv:2601.05740v4 [quant-ph] (2026).

6. G. Li, A. Wu, Y. Shi, A. Javadi-Abhari, Y. Ding, and Y. Xie, "Paulihedral: A Generalized Block-Wise Compiler Optimization Framework for Quantum Simulation Kernels," in *ASPLOS 2022*, 554–569 (2022). DOI: 10.1145/3503222.3507715.